



INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO
ESCOM

Trabajo Terminal

**Plataforma Web para la Denuncia Segura y Prevención de Riesgos
Digitales**

2026-B009

Presentan

Edgar Jair Flores Espinosa

Jose Bryan Omar Jimenez Velazquez

Leilani Shareni Santos Rivera

Directores

M. en C. Ulises Velez Saldaña

Dr. Jessie Paulina Guzmán Flores

Advertencia

“Este documento contiene información desarrollada por la Escuela Superior de Cómputo del Instituto Politécnico Nacional, a partir de datos y documentos con derecho de propiedad y por lo tanto, su uso quedará restringido a las aplicaciones que explícitamente se convengan.”

La aplicación no convenida exime a la escuela su responsabilidad técnica y da lugar a las consecuencias legales que para tal efecto se determinen.

Información adicional sobre este reporte técnico podrá obtenerse en:

La Subdirección Académica de la Escuela Superior de Cómputo del Instituto Politécnico Nacional, situada en Av. Juan de Dios Bátiz s/n Teléfono: 57296000, extensión 52000.

Agradecimientos

A mis padres, por su apoyo incondicional y por ser el pilar fundamental en mi formación académica y personal.

A mis directores, el M. en C. Ulises Vélez Saldaña y la Dra. Jessie Paulina Guzmán Flores, por su invaluable guía, conocimiento y por confiar en este trabajo terminal.

Al Instituto Politécnico Nacional y a la Escuela Superior de Cómputo, por brindarme las herramientas y el entorno necesario para formarme como profesional.

*PONER LO QUE USTEDES QUIERAN, LO DE ARRIBA LO COPIE
JAJAJAJA*

Índice General

1. Introducción	1
1.1. Planteamiento del Problema	2
1.1.1. Contexto y situación problemática	2
1.1.2. Problema a resolver	2
1.2. Solución Propuesta	3
1.3. Justificación	3
1.3.1. Ámbito social e institucional	3
1.3.2. Perspectiva técnica	3
1.4. Objetivos	4
1.4.1. Objetivo General	4
1.4.2. Objetivos específicos	4
1.5. Alcances y Limitaciones	5
1.5.1. Alcances	5
1.5.2. Limitaciones	5
1.6. Estado del Arte	5
1.6.1. Sistemas orientados a la denuncia y gestión de casos	5
1.7. Metodología	5
1.8. Estructura del documento	6
1.9. Nomenclatura	6
2. Fundamentos Tecnológicos del Motor de Clasificación	7
2.1. Problema de clasificación en el contexto de la DDP	7
2.1.1. Taxonomía de denuncias	7
2.1.2. Clasificación multiclase vs multietiqueta	8
2.1.3. Ciclo de vida de los datos en el aprendizaje supervisado	9
2.2. Fundamentos lógicos	10
2.2.1. Inferencia Bayesiana (probabilidad a priori y a posteriori)	10

2.2.2.	Algebra lineal en PLN	11
2.3.	Preprocesamiento avanzado de lenguaje natural (PLN)	14
2.3.1.	Limpieza estructural (normalización de Unicode, eliminación de ruido y stopwords específicas)	14
2.3.2.	Análisis morfosintáctico	14
2.3.3.	Estrategias de tokenización (Unigramas, Bigramas y su impacto en la captura de frases)	17
2.4.	Vectorización (Representación numérica)	17
2.4.1.	Modelos de bolsa de palabras (BoW)	17
2.4.2.	TF-IDF (Term Frequency - Inverse Document Frequency)	19
2.4.3.	Modelos de continuidad (Word embeddings)	20
2.5.	Profundización en el Algoritmo Multinomial Naive Bayes	21
2.5.1.	Supuesto de independencia condicional (ventajas y desventajas)	21
2.5.2.	Estimación de máxima verosimilitud	22
2.5.3.	El problema de la probabilidad cero	23
2.6.	Support Vector Machines	24
2.6.1.	Optimización de hiperplano de separación (margen máximo)	24
2.6.2.	Flexibilidad mediante variables de holgura	25
2.6.3.	El truco kernel	26
2.6.4.	Enfoque matemático	27
2.7.	Estrategias de entrenamiento y validación	30
2.7.1.	Manejo de clases desbalanceadas	30
2.7.2.	Optimización de hiperparámetros	30
2.7.3.	Validación cruzada (K-Fold Cross-Validation)	31
2.8.	Evaluación de la inteligencia del sistema	31
3.	Infraestructura y Tecnologías	33
3.1.	Estudio de Arquitecturas de Software	33
3.1.1.	Introducción a las Arquitecturas de Software	33
3.1.2.	Arquitectura Monolítica	34
3.1.3.	Arquitectura Orientada a Servicios (SOA)	34
3.1.4.	Arquitectura de Microservicios	34
3.1.5.	Arquitectura de Microservicios en 3 Capas	35
3.1.6.	Arquitectura Dirigida por Eventos (Event-Driven)	35
3.1.7.	Arquitectura Serverless (FaaS)	36
3.1.8.	Arquitectura Hexagonal (Ports and Adapters)	36

3.1.9. Comparación de Arquitecturas	37
3.1.10. Selección de Arquitectura	37
4. Requerimientos del Sistema	38
4.1. Requerimientos Funcionales (RF)	38
4.2. Reglas de Negocio (RN)	38
Apéndice Pantallas y Mockups del Sistema	39
.1. Perfil: Quejoso	39
.2. Perfil: Recepcionista	40
.3. Perfil: Oficina de Primer Contacto	41
.4. Perfil: Subdefensores	42
Apéndice Casos de Uso	43

Índice de figuras

2.1.	Representación del espacio tridimensional para el modelo de bolsa de palabras. . .	18
2.2.	Puntos sobre el espacio tridimensional	18
2.3.	Puntos mostrados en un espacio de dos dimensiones	19
2.4.	Gráfica que representa datos no lineales	27
1.	MQ-01: Pantalla de inicio y selección de trámite.	39
2.	Diagrama general de casos de uso del sistema DDP.	43

Índice de tablas

1.1.	Sistemas orientados a la denuncia y gestión de casos.	6
1.	Inventario de Mockups - Perfil Quejoso.	39
2.	Inventario de Mockups - Perfil Recepcionista.	40
3.	Inventario de Mockups - Perfil Primer Contacto.	41
4.	Inventario de Mockups - Perfil Subdefensores.	42

Capítulo 1

Introducción

En el Instituto Politécnico Nacional (IPN), que es una institución reconocida por la educación tecnológica en México, tiene como principios establecidos bajo la Ley Orgánica [1] y su Reglamento Interno [2] el compromiso propio de garantizar un entorno de respeto, equidad y legalidad para todos los integrantes de su comunidad. Ante la identificación de diversas violaciones o vulneraciones a los derechos de estudiantes, docentes y administrativos, el IPN determinó la creación de la Defensoría de los Derechos Politécnicos (DDP) a través de su acuerdo de creación oficial [3]. El cual es un órgano dotado de independencia funcional, que surge como el mecanismo institucional encargado del seguimiento y gestión de las inconformidades derivadas de actos u omisiones que incumplan los derechos Politécnicos.

La gestión de estas quejas representa un proceso crítico que demanda no solo de la sensibilidad humana, sino una administración operativa de alta precisión. Actualmente, la DDP rige su operación mediante el Manual de Organización [4] y el Manual de Procedimientos [5], los cuales establecen las etapas de recepción, radicación, investigación y resolución. No obstante, la ejecución de estos procesos ha presentado desafíos tecnológicos significativos, ya que el flujo de trabajo actual depende de mecanismos manuales y herramientas de oficina descentralizadas que impiden una integración total de la información. Esta carencia de integración tecnológica, sumada al creciente volumen de solicitudes diarias, compromete la eficiencia del personal y extiende los tiempos de respuesta institucionales.

1.1 Planteamiento del Problema

La DDP es el órgano pilar encargado de salvaguardar la legalidad y el respeto a los derechos politécnicos dentro de la comunidad del IPN, la cual integra a estudiantes, docentes y personal administrativo. Su función es esencial para la atención y resolución de quejas; no obstante, ante la creciente demanda de servicios y el tiempo requerido de procesos operativos de los casos actuales, se ha identificado la oportunidad de fortalecer su eficiencia operativa a través de la modernización tecnológica. Bajo el esquema de flujo operativo actual, se presentan tres puntos críticos que fundamentan el desarrollo de la solución planteada:

1.1.1 Contexto y situación problemática

En primer lugar, se identifica la necesidad de optimización de capital humano y la carga operativa, debido a que la atención jurídica sustantiva es realizada por un cuerpo especializado de 5 abogados, quienes gestionan la totalidad de las quejas competentes. La cantidad de quejas recibidas genera una carga administrativa que, al ser gestionada mediante procesos manuales, limita el tiempo disponible para cada abogado, resultando estratégico implementar una solución tecnológica que asista en las tareas repetitivas y permita al personal enfocarse en la resolución de los casos.

1.1.2 Problema a resolver

Derivado de lo anterior, existe un reto en la eficiencia de la recepción y clasificación de solicitudes durante la etapa de *“Primer Contacto”*. Ya que, al recibir información por múltiples canales descentralizados, las solicitudes suelen presentarse de forma incompleta o no son competentes para el órgano de la DDP, resaltando la falta de un asistente tecnológico para el filtrado de competencia, obligando a la revisión manual, retrasando la atención a casos de alta prioridad.

Finalmente, la gestión de documentos realizada de forma tradicional y la falta de seguimiento impactan en la transparencia del proceso. La actual dependencia de métodos de archivo físicos dificulta la consulta rápida de antecedentes. La transición a una plataforma integral no solo brindaría certidumbre al quejoso sobre el estatus de su queja, sino que permitiría al personal de la DDP la revisión de documentos y datos importantes sobre los casos.

1.2 Solución Propuesta

Se propone el desarrollo de una plataforma web integral diseñada para centralizar, automatizar y optimizar el ciclo de vida de las quejas dentro de la DDP. La solución se fundamenta en una arquitectura basada en microservicios y contempla la implementación de un módulo de asistencia inteligente en la etapa de primer contacto, el cual utiliza técnicas de PLN para asistir al usuario en la determinación preliminar de la competencia de su queja. Se permitirá la gestión basada en roles, garantizando que cada actor interactúe con la información de acuerdo con sus facultades normativas, asegurando el seguimiento total del expediente desde su recepción hasta su resolución final.

1.3 Justificación

La implementación de esta plataforma web se justifica por la necesidad estratégica de fortalecer los procesos operativos de la DDP, transformando una gestión actual basada en métodos manuales y herramientas de oficina descentralizadas hacia un flujo digital inteligente.

1.3.1 Ámbito social e institucional

En el ámbito social e institucional, el proyecto busca fortalecer varios puntos empezando por la eliminación de barreras que impiden una mejor comunicación entre la comunidad politécnica y la DDP al proporcionar un canal de seguimiento transparente que elimine la incertidumbre del quejoso. De igual forma, al automatizar las tareas administrativas, se potencia la capacidad de respuesta del personal de la Defensoría y se reduce el costo de gestión operativa, permitiendo que el cuerpo de abogados centralice sus esfuerzos en el análisis jurídico de fondo a las quejas procedentes, apoyando una atención equitativa y eficiente ante la creciente demanda de quejas que reciben diariamente.

1.3.2 Perspectiva técnica

Desde la perspectiva técnica, el desarrollo de este sistema se fundamenta en la adopción de una arquitectura de microservicios permitiendo la oportunidad de escalabilidad y mantenimiento independiente de sus componentes. Ya que la integración de tecnologías como Java Spring Boot y PostgreSQL permite el manejo seguro de información sensible, mientras que la incorporación de un clasificador basado en PLN demuestra una innovación clave para resolver los puntos significativos

desde el área de primer contacto. Esta solución resuelve la fragmentación de datos y fortalece la integridad de un repositorio histórico de quejas para futuras consultas y auditorías.

Finalmente, dado que la plataforma procesa información de carácter personal y sensible dentro del entorno educativo, su diseño integra protocolos de acceso basado en roles y está apegado al cumplimiento legal y ético. Logrando que este enfoque no solo optimice la operación interna de la DDP, sino que consolide una infraestructura digital confiable que otorgue a la DDP eficiencia administrativa apoyando el cumplimiento de sus procesos operativos dentro del IPN.

1.4 Objetivos

1.4.1 Objetivo General

Desarrollar una plataforma web basada en una arquitectura de microservicios mediante la integración de módulos de gestión documental y un motor de procesamiento de lenguaje natural para sistematizar el flujo operativo de recepción, clasificación y seguimiento de quejas de la DDP.

1.4.2 Objetivos específicos

- Modelar las etapas de proceso y resolución de quejas con base en las reglas de negocio del Manual de Procedimientos de la DDP para automatizar la transición de estados del expediente y el cumplimiento de términos legales.
- Diseñar una arquitectura de microservicios desacoplada empleando el ecosistema de Java Spring Boot y comunicación vía API REST para garantizar la independencia operativa entre el sistema de gestión administrativa y el módulo de inteligencia artificial.
- Implementar un clasificador de texto neuro-simbólico utilizando Word Embeddings y sistemas basados en reglas para determinar la competencia de las solicitudes desde el primer contacto y reducir la carga de clasificación manual.
- Construir un repositorio digital de evidencias y expedientes mediante el sistema de gestión de bases de datos PostgreSQL para centralizar la documentación soporte y asegurar la integridad de la información jurídica manejada por la Defensoría.
- Desarrollar un módulo de consulta y seguimiento para el quejoso a través de un sistema de folios digitales y notificaciones de estatus para brindar certidumbre sobre el avance del proceso

y reducir la saturación de los canales de comunicación presenciales.

1.5 Alcances y Limitaciones

1.5.1 Alcances

La plataforma web permitirá el registro, seguimiento y gestión integral de las quejas recibidas por la DDP, abarcando desde el primer contacto hasta la resolución final del expediente. El sistema implementará un clasificador basado en PLN para asistir en la determinación de competencia de las solicitudes, así como un repositorio digital para el almacenamiento seguro de documentos y evidencias. Se desarrollarán módulos específicos para cada rol institucional (Abogado Asesor, Subdefensor, Área Secretarial, Área de Primer Contacto y Titular) y se proporcionará un sistema de consulta pública mediante folios digitales.

1.5.2 Limitaciones

El sistema no sustituirá el criterio jurídico de los abogados de la DDP, sino que fungirá como una herramienta de apoyo en la clasificación inicial. La plataforma no realizará resoluciones automáticas de quejas ni emitirá dictámenes legales. Asimismo, el clasificador basado en IA requerirá un período de entrenamiento y ajuste con datos reales para alcanzar niveles óptimos de precisión, por lo que en etapas iniciales operará de manera asistida con supervisión humana.

1.6 Estado del Arte

1.6.1 Sistemas orientados a la denuncia y gestión de casos

1.7 Metodología

Para el desarrollo de este proyecto se ha decidido utilizar un enfoque ágil híbrido: **Scrum-ban**. Combina la estructura iterativa y planificación de Scrum con la gestión visual y flexible de Kanban. Esta decisión permite mantener un flujo constante evitando bloqueos y garantiza entregas parciales de valor mediante Sprints planificados.

Plataforma	Descripción
BullyButton.com	Herramienta de pago para reportar incidentes de acoso y genera estadísticas [7].
Ventanilla Escolar	Plataforma mexicana para orientación sobre incidentes escolares [8].
Alerta IPN	Herramienta institucional para reportar sucesos anómalos [9].
UNAM - Denuncia Línea	Plataforma de la UNAM para registro de quejas normativas [10].
PROFEDET Digital	Sistema para la defensa de derechos laborales [11].

Tabla 1.1: Sistemas orientados a la denuncia y gestión de casos.

1.8 Estructura del documento

Este documento va dirigido a los directores del Trabajo Terminal, el M. en C. Ulises Vélez Saldaña y la Dra. Jessie Paulina Guzmán Flores. Se estructura de la siguiente manera: el Capítulo 2 presenta el modelado del negocio; el Capítulo 3 describe los requerimientos; el Capítulo 4 detalla la arquitectura; el Capítulo 5 presenta el diseño de interfaz; y el Capítulo 6 expone las conclusiones.

1.9 Nomenclatura

La información se estructura mediante estándares de ingeniería de software para la documentación de requerimientos, utilizando diagramas UML y tablas descriptivas para garantizar la claridad en la especificación de componentes.

Capítulo 2

Fundamentos Tecnológicos del Motor de Clasificación

Para cumplir con uno de los objetivos específicos que establece la implementación de un clasificador de texto con el fin de identificar la competencia o no competencia a la Defensoría de Derechos Politécnicos para así automatizar el flujo operativo de recepción, clasificación y seguimiento, para el entrenamiento de este clasificador se usará un modelo de aprendizaje supervisado como SVM (Máquina de Soporte Vectorial) y Naive Bayes como modelo de clasificación probabilístico dentro del aprendizaje automático supervisado.

2.1 Problema de clasificación en el contexto de la DDP

La Defensoría de Derechos Politécnicos no presenta conflictos al clasificar la competencia de una queja al departamento, pues esta competencia está dada por el factor humano (abogados) quienes por medio de parámetros y al artículo 13 del acuerdo de creación detectan la competencia de la queja.

2.1.1 Taxonomía de denuncias

Entender la clasificación de las quejas y el porqué es fundamental para la organización de los datos al momento de entrenar el modelo. Esta es una clasificación binaria, pues solo encontramos dos opciones: competente o no competente.

Sin embargo, existen diversas formas de catalogar una queja. Este catálogo es usado para filtrar el

motivo de la queja con el fin de organizar los hechos presentados y, por ende, realizar de manera más eficiente el trabajo de los abogados. La competencia se define por:

1. **Identificación del Sujeto (Legitimación):** Se verifica que el quejoso sea un miembro de la comunidad politécnica (alumno con número de cuenta o empleado con número de trabajador) y que la autoridad señalada pertenezca a una unidad académica o administrativa del IPN.
2. **Temporalidad (Plazo de 90 días):** La queja debe interponerse dentro de los noventa días siguientes a los hechos que la motivan. Si excede este tiempo, se clasifica automáticamente como improcedente.
3. **Naturaleza del Conflicto (Exclusiones):** La competencia se descarta si el hecho es de carácter anónimo o si versa sobre materias ajenas a la Defensoría, específicamente:
 - 3.1. **Asuntos Laborales:** Conflictos obrero-patronales o sindicales que no entran en la esfera de derechos politécnicos.
 - 3.2. **Evaluaciones Académicas:** Inconformidades puramente sobre criterios de calificación, a menos que impliquen una vulneración de derechos humanos o politécnicos.
 - 3.3. **Resoluciones de otras instancias:** Casos que ya estén siendo atendidos o hayan sido resueltos por el Órgano Interno de Control o el Abogado General.
4. **Formalidad del Acto:** Se requiere que la queja contenga la descripción detallada de los actos, la firma o huella del promovente y, en caso de menores de edad, la validación del tutor.
5. **Análisis de Reincidencia:** Aunque la queja sea nueva, el factor humano evalúa si el "afectador" (autoridad o docente) presenta un patrón de conducta registrado en otros expedientes del sistema, lo que ayuda a catalogar la gravedad y urgencia del asunto.

2.1.2 Clasificación multiclase vs multietiqueta

La clasificación inicial es multiclase específicamente en su primer nivel es binaria, pues el estatus jurídico de una queja es excluyente en los siguientes puntos:

- Acorde al artículo 13, una queja no puede ser simultáneamente "Competente" e "Improcedente".
- El sistema debe arrojar un veredicto definitivo para procesar el folio, o se admite o se rechaza.
- Dado que la competencia se define por la ausencia de improcedencia, el modelo debe asignar la queja a una sola categoría.

Una vez declarada la queja como competente, el sistema se beneficia de un enfoque multietiqueta para el análisis de los hechos:

- Un solo relato de hechos puede contener múltiples vulneraciones. Por ejemplo, una queja puede catalogarse simultáneamente como "Violencia de Género", "Abuso de Autoridad" y "Afectación Académica".
- El uso de etiquetas múltiples permite al sistema detectar patrones más complejos. Si un docente tiene etiquetas recurrentes de "Acoso verbal" en diferentes expedientes, el sistema alerta sobre un escalamiento de violencia.
- Permite a los abogados filtrar casos por materias específicas para aplicar los mismos criterios de lógica y legalidad en la valoración de pruebas similares.

Si bien el diseño de una arquitectura híbrida representa la solución óptima, para este sistema se ha optado por un enfoque de clasificación multiclase (binaria). Esta decisión se fundamenta en la viabilidad técnica: la implementación de una clasificación multietiqueta requeriría un volumen de datos de entrenamiento con el que no se cuenta actualmente. En contraste, el enfoque binario permite concentrar la muestra en solo dos categorías, garantizando una mayor densidad de datos por clase y, por ende, una mayor robustez en el entrenamiento del modelo.

2.1.3 Ciclo de vida de los datos en el aprendizaje supervisado

El ciclo de vida de los datos en este proyecto no es solo un proceso técnico, sino que debe alinearse estrictamente con los lineamientos del Artículo 13 del Reglamento de la Defensoría. Este ciclo garantiza que la información sea útil para entrenar los modelos de SVM y Naive Bayes mencionados anteriormente:

1. **Captura de datos:** Los datos se originan cuando el quejoso interactúa con el sistema digital. Para cumplir con la Fracción II del Artículo 13, el sistema captura de forma estructurada el nombre, número de cuenta/empleado, la autoridad imputada y la descripción detallada de los hechos.
2. **Procesamiento y Filtrado:** El sistema calcula automáticamente si la queja se encuentra dentro del plazo de 90 días naturales desde el hecho, marcando esta variable como crítica para la clasificación de improcedencia. La descripción de los actos se somete a técnicas de Procesamiento de Lenguaje Natural (PLN) para identificar palabras clave relacionadas con las exclusiones de competencia (ej. "sindicato", "evaluación", "laboral").

3. **Etiquetado (Supervisión):** Dado que es un aprendizaje supervisado, el modelo se entrena con el histórico de decisiones de los abogados. Cada expediente histórico se etiqueta como "Competente" (Admitida) o "No Competente" (Rechazo fundado y motivado) basándose en si cumplió con los lineamientos del Artículo 13.
4. **División de Dataset:** Debido a la escasez de datos para etiquetas múltiples, se utiliza el conjunto de datos para fortalecer la clasificación binaria. Se divide la muestra en datos de entrenamiento (para que el modelo aprenda la correlación entre hechos y competencia) y datos de validación (para asegurar que el modelo pueda predecir correctamente la competencia de quejas nuevas).
5. **Retroalimentación:** El modelo sugiere una clasificación al abogado. Si el factor humano corrige la sugerencia del sistema (por ejemplo, admitiendo una queja que el modelo sugirió rechazar), este dato se reincorpora al ciclo para re-entrenar el modelo y mejorar su precisión futura.
6. **Anonimización y Resguardo:** Los datos permanecen activos para la detección de patrones de reincidencia. Al cumplirse el límite de 5 años de resguardo físico, la información digital puede permanecer anonimizada para fines estadísticos y de mejora continua del motor de clasificación.

2.2 Fundamentos lógicos

2.2.1 Inferencia Bayesiana (probabilidad a priori y a posteriori)

La inferencia bayesiana es un método estadístico el cual permite actualizar la probabilidad de un evento a medida que se obtiene nueva evidencia o información, sigue siendo el Teorema de Bayes, indica cuál es la probabilidad de que desarrolle un determinado escenario, teniendo en cuenta un mínimo de dos alternativas, pues tienen que existir opciones en las que elegir, la formula prioriza tres conceptos:

1. **Priorización y posterización:** En primer lugar, se utiliza una probabilidad previa que se actualizara a través de dato, de esta manera obtenemos probabilidad posterior, pues la idea es comprobar ese contraste.
2. **Verosimilitud:** Son datos que tienen que ser verdaderos. Si los datos no son verosímiles, se tendrán que consultar los sistemas de capacitación de los estos.

3. **Actuación iterativa:** Cuando más información se tenga, las probabilidades posteriores pueden ser previas.

Probabilidad a priori es la probabilidad que se asigna a un evento antes de observar cualquier evidencia o realizar un experimento. Representa el conocimiento inicial o creencia sobre la posibilidad de un suceso, que puede basarse en información subjetiva, objetiva o una mezcla de ambas.

Probabilidad a posteriori es la probabilidad de un evento después de haber observado evidencia o realizado un experimento. Se calcula utilizando el Teorema de Bayes, que actualiza la probabilidad a priori con nueva información. Por ejemplo, si se sabe que una persona dio positivo en una prueba de enfermedad, la probabilidad a posteriori de que realmente tenga la enfermedad se calcula considerando la precisión de la prueba y la prevalencia de la enfermedad en la población.

La relación se expresa mediante el Teorema de Bayes:

$$P(H|D) = \frac{P(D|H) \cdot P(H)}{P(D)} \quad (2.1)$$

En este caso, debemos decir que $P(H|D)$ es el resultado final de la probabilidad posterior de la hipótesis H a partir de la evidencia D . Por otra parte, $P(D|H)$ es la verosimilitud; $P(H)$ la probabilidad previa y $P(D)$ la probabilidad posterior.

Esta fórmula se puede incluir en numerosos elementos y, también, cuando se diseñan modelos. Por eso este teorema, cuyo origen está en 1763, sigue teniendo especial importancia en las nuevas tecnologías.

2.2.2 Álgebra lineal en PLN

Para que un modelo de aprendizaje procese el lenguaje humano, los textos se deben transformar en representaciones numéricas, por ende, el álgebra lineal proporciona la estructura para manipular estas representaciones en espacios vectoriales, donde un espacio vectorial es la representación de un conjunto de documentos como vectores en un espacio común y es fundamental para una serie de operaciones de recuperación de información, desde la puntuación de documentos en una consulta, así como clasificación y agrupamiento.

Este modelo posibilita determinar la importancia de un término en un documento específico y entiende que los documentos pueden expresarse en función de estos vectores que recogen la frecuencia en que aparecen los términos en los documentos. Los términos que formaran esta matriz

serán términos no vacíos, esto significa que contienen significado para el momento de recuperar información, teniéndolos almacenados en formato stemmed (reducidos los términos a una raíz común).

Este procedimiento se puede dividir en tres etapas:

1. La primera es la **etapa de indexación de documentos** donde los términos que contienen información se extraen del texto de documento. Muchas de las palabras no describen el contenido, como los artículos y preposiciones, las palabras no significativas se eliminan del vector del documento, de esta manera, el documento solo se representará por palabras que contengan contenido representativo. Esta indexación puede basarse en la frecuencia de los términos, donde los términos tienen tanto alta como baja frecuencia dentro de un documento se consideran palabras funcionales. Para lograr hacer esta etapa de manera funcional, se hace uso de una lista de palabras comunes que se usan de manera frecuente, para así eliminar estas palabras que provocarían errores (stopwords), normalmente, el 40-50 % del número total de palabras se elimina con ayuda de la lista de detención.
2. La segunda etapa es la **ponderación de los términos** para mejorar la recuperación de documentos relevantes para el usuario, esta ponderación temporal se explica mediante la relación con el recuerdo y la especificidad con la precisión, este término (ponderación) se basa completamente en estadísticas de un solo término. Hay tres factores para la ponderación del término: factor de término, factor de frecuencia de recopilación y factor de normalización de longitud.
3. La última etapa **clasifica el documento** con respecto a la consulta según una medida de similitud, esta se determina mediante el uso de coeficientes asociativos basados en el producto interno del vector de documento y el vector de consulta. La medida de similitud es el coeficiente del coseno, que mide el ángulo entre un vector de documento y el vector de consulta.

Espacios de características (N-dimensiones)

Un espacio de características es una construcción matemática donde cada palabra única (después del proceso de stemming y eliminación de stopwords) se convierte en un eje o dimensión independiente.

Definimos las dimensiones si el vocabulario total de todas las quejas históricas de la DDP contiene V términos únicos, entonces, cada queja se representa con un vector x en un espacio de dimensiones V , denotado como $\mathbb{R}^V$.

Entonces, una queja específica se define como:

$$\vec{x} = (w_1, w_2, w_3, \dots, w_V) \quad (2.2)$$

Donde w_i representa el peso del término i en esa queja. Si una palabra x tiene un peso alto en el vector, este apuntará hacia un reglón del espacio asociada a la improcedencia. Dado que el lenguaje es vasto, el espacio de características suele ser de alta dimensionalidad, lo que significa que la mayoría de los componentes del vector serán cero, ya que una sola queja no contiene todas las palabras.

Métricas de proximidad

Una vez que las quejas están proyectadas en el espacio de características, el motor de clasificación deberá medir que tan cerca está una queja nueva de los ejemplos etiquetados como “Competentes” o “No competentes”.

La métrica principal utilizada es la Similitud de Coseno. A diferencia de la distancia euclidiana (que mide la separación en línea recta), el coseno mide el ángulo entre dos vectores. Esto es crítico para la DDP porque:

- **Independencia de la longitud:** Una queja muy extensa y una muy breve que traten el mismo tema (ej. “evaluación académica”) tendrán vectores que apuntan en la misma dirección, aunque su longitud sea distinta.
- **Formalismo Matemático:** La similitud entre una queja A y una queja B se calcula como:

$$\cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}} \quad (2.3)$$

- Un valor de 1 indica que las quejas son idénticas en temática.
- Un valor de 0 indica que no comparten ninguna palabra clave relevante.

Esta métrica es la base para que algoritmos como SVM puedan trazar una frontera (hiperplano) que divida eficientemente ambos grupos de quejas.

2.3 Preprocesamiento avanzado de lenguaje natural (PLN)

Como hemos visto, la implementación de lenguaje natural comienza con la recopilación y preparación de datos de texto, no estructurados, pero un paso que es crucial en el desarrollo de PLN, ha sido el preprocesamiento de información.

El preprocesamiento transforma el lenguaje natural desestructurado en datos aptos para los algoritmos SVM y Naive Bayes, esto es esencial en todos los modelos, sin embargo, en la DDP, este paso es crítico para filtrar la narrativa emocional y conservar las características que definen la competencia al departamento.

2.3.1 Limpieza estructural (normalización de Unicode, eliminación de ruido y stopwords específicas)

La limpieza de datos es fundamental, pues los textos pueden contener ruido y elementos irrelevantes, existen técnicas básicas como la eliminación de puntuación, la conversión de mayúsculas o minúsculas y la eliminación de palabras vacías.

La normalización es una de ellas, la cual busca estandarizar el texto para facilitar su análisis. Esto permite convertir abreviaturas y jerga a palabras completas, mapear sinónimos y homónimos, así podemos resolver ambigüedades lingüísticas, así removemos caracteres especiales que no aportan un significado.

Y para esto, el uso de Stop Words es necesario, pues estas son palabras que no aportan algún significado por sí solas, y suelen ser ignoradas por los motores de búsqueda. Estas palabras se eliminan durante el análisis de texto, así podemos mejorar la eficiencia, reducimos el ruido y enfocamos el análisis en palabras clave más relevantes.

2.3.2 Análisis morfosintáctico

El análisis morfosintáctico se centra en el estudio de la forma de cada palabra, así determinamos que clase de palabra es. Este es una combinación entre el análisis sintáctico y el análisis morfológico, entonces, para entenderlo, debemos de conocer los dos conceptos anteriores.

Análisis sintáctico: en este buscamos señalar cuáles son las funciones sintácticas que cada una de las palabras tiene dentro de la oración que vamos a analizar.

Análisis morfológico: es aquel en el que nos centramos en la clase, forma o categoría de las palabras que forman la oración.

Eso indica que el análisis morfosintáctico es aquel que combina las dos formas anteriores y por tanto el más complejo de los dos y para realizar este, debemos de realizar un análisis morfológico, para comprender y señalar los tipos y clases de palabras, tomaremos el siguiente ejemplo:

Enrique explicó la historia a su prima

Lo primero que debemos hacer es señalar debajo de cada una de las palabras a qué tipo pertenecen quedando de la siguiente forma:

- **Enrique:** Nombre propio, masculino singular.
- **Explicó:** Verbo explicar, tercera persona del singular del presente de Indicativo.
- **A:** Preposición.
- **Su:** Determinante posesivo.
- **Prima:** Nombre común, femenino singular.

En el análisis sintáctico es el que nos muestra funciones que tienen cada una de las palabras dentro de una oración, así podemos distinguir las oraciones en dos partes:

- **Sujeto:** es la persona, animal o cosa que padece la acción del verbo. Está formado por un sintagma nominal, en el que el nombre es el núcleo de dicho sujeto.
- **Predicado:** nos muestra la acción del verbo y los complementos que le acompañan. Está formado por un sintagma verbal en el que el núcleo siempre será el verbo.

Dentro del predicado también existen otras palabras que también tienen una función dentro de la oración: "la historia" que es el Complemento Directo y "a su prima" que es el Complemento Indirecto. Así podemos señalar que dentro del predicado nos encontramos con que la palabra explicó es el núcleo.

Por lo tanto, el análisis morfosintáctico consiste en señalar todas las partes que componen la oración, las funciones que cumplen cada una de ellas, los tipos de palabras y cómo estas se comportan dentro de la misma. Este proceso es fundamental para tareas avanzadas como la traducción automática, la extracción de información, el reconocimiento de entidades y la generación de texto, ya que permite una comprensión profunda del significado y las relaciones entre palabras en un texto.

Stemming

Es el proceso de reducir la forma flexiva de una palabra a una denominada “base”. Es uno de los métodos principales que reduce las variantes flexivas dentro de un conjunto de datos de texto a un lexema morfológico. Con ello, mejoramos el procesamiento de texto en los sistemas de aprendizaje automático y de recuperación de información.

Las máquinas, desde las funciones de búsqueda y localización hasta los modelos de aprendizaje profundo, procesan el lenguaje en gran medida según la forma, y muchos investigadores sostienen que las computadoras no pueden entender el significado del lenguaje. Sin embargo, es un hecho que los modelos de aprendizaje automático deben ser entrenados para reconocer diferentes palabras como variantes morfológicas de una palabra base. Por ejemplo, en los motores de búsqueda, los usuarios pueden enviar una consulta con una palabra (por ejemplo, invirtiendo), pero esperan resultados que utilicen cualquier forma flexiva de la palabra (por ejemplo, invertir, inversión, inversiones, etc.).

Para muchas tareas de minería de textos, entre ellas, la clasificación de textos, la agrupación en clústeres, la indexación y otras, el stemming ayuda a mejorar la precisión al reducir la dimensionalidad de los algoritmos de aprendizaje automático y agrupar palabras según el concepto. De este modo, el stemming sirve como un paso importante en el desarrollo de modelos de lenguaje grandes.

Stemming es una etapa en el pipeline de la minería de textos en la que los datos de texto sin procesar se convierten en un formato estructurado para su procesamiento automático. Esencialmente, el stemming elimina los afijos de las palabras, dejando solo la forma base. Si un quejoso escribiera profesor, profesores o profesora, el algoritmo los reduciría a la raíz común profe.es.

Lematización

La lematización es la tarea de reducir las variantes morfológicas a una forma base de diccionario. Mientras el Stemming simplemente elimina los sufijos comunes del final de los tokens de palabras, la lematización garantiza que la palabra de salida sea una forma estandarizada existente de la palabra que se puede encontrar en el diccionario.

Debido a que la lematización tiene como objetivo generar formas basadas en diccionarios, requiere un análisis morfológico más estable que el stemming. El etiquetado de parte del discurso (POS) es un paso crucial en la lematización. En la DDP la precisión terminológica es fundamental; mientras el stemming podría cortar la palabra “ley” de forma impredecible, la lematización identifica que

“leyes” tiene como lema “ley” y que “fueron” tiene como lema “ser”. Es más lenta que el stemming porque requiere una base de datos lingüística, pero garantiza que el "sentido legal" de los hechos presentados en la queja se mantenga intacto.

2.3.3 Estrategias de tokenización (Unigramas, Bigramas y su impacto en la captura de frases)

Para un funcionamiento adecuado del clasificador de la DDP, no basta con analizar palabras aisladas pues la estructura de la frase aporta el contexto de la competencia.

Existen los **Unigramas**, los cuales se generan tomando cada palabra como token independiente, así podemos detectar temas generales, pero pierde el contexto. A diferencia de los **Bigramas**, donde se toman pares de palabras adyacentes, con esto logramos capturar conceptos compuestos que perderían su valor si se separan. Al combinar unigramas y bigramas, el modelo puede distinguir entre una situación y otra, incrementando la precisión al capturar la semántica de una queja.

2.4 Vectorización (Representación numérica)

Es un proceso que se utiliza para asignar palabras en el espacio de vectores, pues un espacio de vectores representa cada palabra con un vector de números reales, también permite que las palabras con significados similares tengan representaciones similares.

2.4.1 Modelos de bolsa de palabras (BoW)

La caracterización de bolsa de palabras puede ser considerado una forma de procesamiento de texto sencilla por su aparente simplicidad en recuento de palabras en un conjunto de textos.

En bolsa de palabras, cada palabra individual se convierte en una dimensión separada del espacio vectorial. Si un conjunto de texto tiene n cantidad de palabras, el espacio vectorial resultante tendrá n dimensiones. El modelo trazará cada documento de texto por separado como un punto en el espacio vectorial. La posición de un punto está determinada por la cantidad de veces que aparece la palabra de esa dimensión dentro del documento del punto.

Supongamos que tenemos un conjunto de textos en el que los contenidos de dos documentos separados son:

- Documento 1: Una rosa es roja, una violeta es azul.
- Documento 2: Mi amor es como una roja, roja rosa.

Para esta ejemplificación usaremos un espacio vectorial de tres dimensiones para *rojo*, *rosa* y *violeta*.

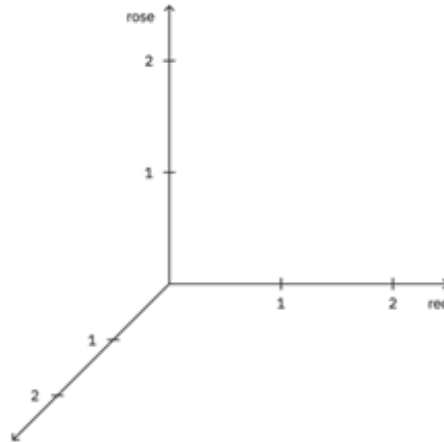


Figura 2.1: Representación del espacio tridimensional para el modelo de bolsa de palabras.

Dado que rojo, rosa y violeta aparecen una vez en el documento 1, el vector para ese documento será $(1,1,1)$. En el documento 2, rojo aparece dos veces, rosa aparece una vez y violeta no aparece. Por lo tanto, el punto vectorial para el documento 2 es $(2,1,0)$.

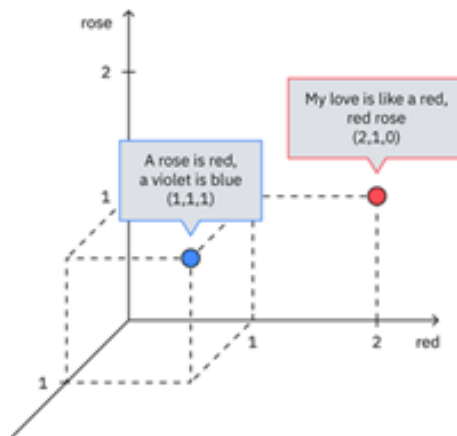


Figura 2.2: Puntos sobre el espacio tridimensional

Para un modelo de bolsa de palabras, lo único que importa es la cantidad de ocurrencias. Esto nos deja como limitación que todas las palabras tienen la misma importancia.

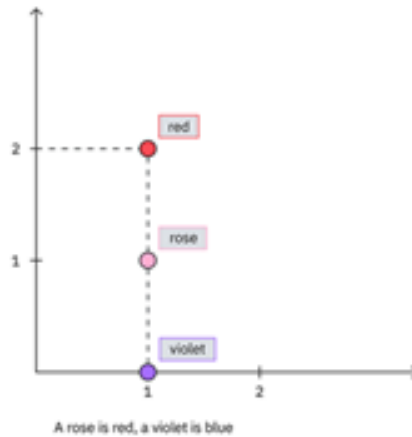


Figura 2.3: Puntos mostrados en un espacio de dos dimensiones

2.4.2 TF-IDF (Term Frequency - Inverse Document Frequency)

Para resolver las limitaciones de BoW, se utilizaría un esquema de pesaje TF-IDF, asignando un valor a cada termino basado no solo en la frecuencia, sino en la relevancia global. Combina dos componentes:

- **Term Frequency (TF):** Mide la frecuencia con la que aparece una palabra en un documento. Una frecuencia más alta sugiere mayor importancia, entonces, si un termino aparece con frecuencia en un documento, es probable que sea relevante para el contenido del documento.

$$TF(t, d) = \frac{\text{Número de veces que el término } t \text{ aparece en el documento } d}{\text{Total de términos en el documento } d} \quad (2.4)$$

- **Inverse Document Frequency (IDF):** Reduce el peso de las palabras comunes en varios documentos y al mismo tiempo aumenta el peso de las palabras raras. Si un termino aparece en menos documentos, es más probables que sea significativo.

$$IDF(t, D) = \log \left(\frac{\text{Número total de documentos en el corpus } D}{\text{Número de documentos que contienen el término } t} \right) \quad (2.5)$$

Este equilibrio permite resaltar términos que son frecuentes dentro de un documento específico, lo que lo convierte en una herramienta útil para tareas de clasificación de texto.

2.4.3 Modelos de continuidad (Word embeddings)

Es una técnica que permite a las máquinas representar el significado de las palabras capturando relaciones complejas entre ellas. Un ejemplo básico sería la palabra “banco” pues podría referirse a una institución bancaria o a un asiento, dependiendo del contexto en el que se encuentre.

Con los Word embeddings asignamos a cada palabra un vector en un espacio de varias dimensiones, con la posición de estos vectores en el espacio observaríamos la cercanía semántica entre las palabras, por ende, si dos palabras tienen significados similares, sus vectores estarán cercanos.

La manera en que se crean los word embeddings consiste en entrenar un modelo en un gran corpus de texto, para así aprender la estructura del lenguaje, esto implica hacer un preprocesamiento en el corpus, en donde tokenizamos las palabras y normalizamos el texto.

Después de haber preprocesado el texto, es necesario hacer uso de una técnica conocida como “ventana de contexto deslizante” para extraer información. Con esto, cada palabra objetivo, se toman en cuenta las palabras que la rodean dentro de un cierto rango, por ende, el modelo será entrenado para aprender a predecir una palabra objetivo usando las palabras de su contexto. .

Word2Vec y GloVe (captura de relaciones semánticas mediante contextos locales)

Son dos modelos que generan representaciones de palabras (word embeddings), utilizan redes neuronales para aprender el contexto de palabras. Word2Vec es un modelo predictivo que a través de redes neuronales aprende vectores de palabras, se basa en el contexto local de las palabras, ya sea prediciendo palabras de contexto dada una palabra central o viceversa, con este enfoque, capturamos relaciones semánticas y sintácticas finas.

GloVe es un modelo basado en conteos, que construye una matriz global de co-ocurrencia que refleja cuantas veces aparecen pares de palabras juntas en todo el corpus. Luego factoriza la matriz para obtener vectores que preservan relaciones estadísticas globales, explícitamente, permite capturar mejor estructuras lineales y relaciones semánticas amplias.

Mientras que TF-IDF solo busca coincidencias exactas de palabras, los embeddings permiten que el sistema entienda que una queja sobre "violencia docente.^{es} semánticamente similar a una de "maltrato de profesor", incluso si no comparten las mismas palabras exactas.

2.5 Profundización en el Algoritmo Multinomial Naive Bayes

Naive Bayes es un modelo bayesiano, es decir, probabilístico. Esto quiere decir que, las predicciones se basan en calcular probabilidades.

Además, tal como su nombre indica, es un modelo Naive, es decir, ingenuo. Esto es porque Naive Bayes asume que el efecto de una variable es independiente al resto de variables. De esta forma, el cálculo de las probabilidades es mucho más sencillo.

En definitiva, si en un mensaje aparece la palabra «buenos» esto será independiente de que aparezca la palabra «días». Como puedes esperar, en la realidad esto no es así. Sin embargo, esta asunción facilita mucho los cálculos, tal como veremos más adelante.

Y es que, en términos matemáticos, el cálculo matemático de Naive Bayes se traduce en la siguiente fórmula, la cual se conoce como regla de Bayes:

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)} \quad (2.6)$$

Donde:

- $P(A|B)$ representa la probabilidad de que ocurra A sabiendo que B ya ha ocurrido.
- $P(B|A)$ es la probabilidad de que ocurra B sabiendo que se ha dado A.
- $P(A)$ Probabilidad de que ocurra A.
- $P(B)$ Probabilidad de que ocurra B.

2.5.1 Supuesto de independencia condicional (ventajas y desventajas)

El supuesto central de los clasificadores Naive Bayes es la independencia condicional de las características dada la clase. Esto significa que, una vez conocida la clase, el valor de una característica no depende del valor de cualquier otra característica. Esto significa que:

$$P(x_1, \dots, x_d|y) = \prod_{i=1}^d P(x_i|y) \quad (2.7)$$

Este supuesto simplifica enormemente el cálculo de la probabilidad conjunta, haciendo que el modelo sea computacionalmente eficiente y fácil de entrenar, incluso con grandes cantidades de datos.

- **Ventajas:** El modelo es rápido de entrenar e inferir, lo que lo hace ideal para tareas como filtrado de spam o clasificación de texto.
- **Limitaciones:** En la práctica, las características rara vez son verdaderamente independientes. Por ejemplo, en un correo electrónico, la palabra "oferta" puede estar relacionada con "descuento". A pesar de esto, el clasificador suele funcionar bien en muchos escenarios reales, especialmente con tamaños de muestra pequeños.

En la práctica, las características rara vez son verdaderamente independientes. Por ejemplo, en un correo electrónico, la palabra "oferta" puede estar relacionada con "descuento". A pesar de esto, el clasificador suele funcionar bien en muchos escenarios reales, especialmente con tamaños de muestra pequeños.

2.5.2 Estimación de máxima verosimilitud

La Estimación de máxima verosimilitud (MLE) en el clasificador Naive Bayes es un método para estimar los parámetros del modelo, como las probabilidades a priori de las clases y las verosimilitudes de las características dadas una clase, basándose en las frecuencias observadas en el conjunto de entrenamiento.

- **Prioris de clase:** $P(c) = \frac{\text{número de registros de clase } c}{\text{número total de registros}}$.
- **Verosimilitud:** Para atributos discretos se estima como frecuencia relativa. Para continuos, se asume distribución normal.

Este enfoque de MLE es directo y eficiente, pero tiene un problema crítico: si una combinación de clase y característica no aparece en el conjunto de entrenamiento, su probabilidad se estima como cero, lo que puede anular toda la predicción. Para evitar esto, se utilizan técnicas de suavizado (como el suavizado de Laplace), que añaden un valor pequeño (por ejemplo, 1) al numerador y al denominador para evitar ceros.

2.5.3 El problema de la probabilidad cero

El problema de la probabilidad cero en Naive Bayes surge cuando una característica (por ejemplo, una palabra en texto) no aparece en los datos de entrenamiento para una clase determinada. Esto hace que la probabilidad condicional de esa característica dada la clase sea cero. Como Naive Bayes multiplica todas las probabilidades condicionales para obtener la probabilidad posterior, un valor cero anula completamente el resultado, llevando a predicciones incorrectas o imposibles, incluso si otras características indican claramente la clase correcta.

Este problema es común en tareas como el filtrado de spam, donde una palabra nueva o rara en el conjunto de entrenamiento puede hacer que el modelo no pueda clasificar un mensaje. Por ejemplo, si la palabra "Comida" nunca aparece en correos de spam, su probabilidad condicional será cero, y cualquier mensaje que la contenga será clasificado erróneamente como "No Spam", aunque otras palabras sugieran lo contrario.

La técnica más utilizada para resolver este problema es el suavizado de Laplace (también llamado suavizado aditivo). Consiste en sumar un valor pequeño (generalmente 1) al recuento de cada combinación de clase y característica, asegurando que ninguna probabilidad sea cero. Esto permite que el modelo haga predicciones incluso para características nuevas o raras.

La fórmula ajustada para el suavizado de Laplace se expresa de la siguiente manera:

$$\hat{P}(x_i|c) = \frac{N_{x_i,c} + \alpha}{N_c + \alpha d} \quad (2.8)$$

Donde:

- $N_{x_i,c}$ es el recuento de la característica x_i en la clase c .
- N_c es el total de características en la clase c .
- d es el número de características posibles (tamaño del vocabulario).
- α es el parámetro de suavizado (comúnmente se utiliza $\alpha = 1$).

Este ajuste mejora la generalización del modelo y evita que el clasificador se rompa ante datos no vistos en el entrenamiento.

2.6 Support Vector Machines

Una Support Vector Machine es un algoritmo de Machine Learning supervisado que se suele utilizar para problemas de clasificación y regresión en aplicaciones de procesamiento de señales, procesamiento del lenguaje natural (PLN), y reconocimiento del habla e imágenes. El objetivo del algoritmo de SVM es encontrar un hiperplano que, en la mejor medida posible, separe los puntos de datos de una clase de los de otra clase. Este hiperplano puede ser una línea para espacio en 2D o un plano para un espacio con n dimensiones, donde n es el número de características de cada observación del conjunto de datos. Pueden existir múltiples hiperplanos que separen las clases de los datos. El hiperplano óptimo, derivado por el algoritmo de SVM, es el que aumenta el margen entre las dos clases.

El margen es la anchura máxima de la franja paralela al hiperplano que no contiene puntos de datos en su interior. Los puntos de datos que marcan el límite de esta franja paralela y están más cerca del hiperplano separador son los vectores de soporte. Los vectores de soporte hacen referencia a un subconjunto de las observaciones de entrenamiento que determinan la ubicación del hiperplano separador.

2.6.1 Optimización de hiperplano de separación (margen máximo)

La Máquina de Vectores de Soporte (SVM) busca encontrar el hiperplano de separación óptimo que maximice el margen entre las clases; es decir, la distancia entre el hiperplano y los puntos de datos más cercanos de cada clase, conocidos como **vectores de soporte**.

El objetivo del problema de optimización en SVM se formula como:

- **Maximizar el margen:** Equivalente a minimizar $\|w\|$, donde w es el vector normal al hiperplano.
- **Sujeto a las restricciones:** Para cada dato (x_i, y_i) , se debe cumplir que $y_i(\langle w, x_i \rangle + b) \geq 1$, donde $y_i \in \{-1, +1\}$ es la etiqueta de clase.

Este problema se resuelve mediante optimización dual usando multiplicadores de Lagrange, lo que permite obtener la solución mediante el lagrangiano dual, donde solo los vectores de soporte (aquellos con $\alpha_i > 0$) influyen en el hiperplano final. Para datos no linealmente separables, se introduce una variable de holgura ξ_i y un parámetro de regularización C , que controla el *trade-off* entre:

- **Máximo margen:** Menor complejidad y mayor capacidad de generalización.
- **Menos errores de clasificación:** Mayor ajuste al conjunto de entrenamiento.

El hiperplano final de decisión se define mediante la siguiente función:

$$f(x) = \text{sign} \left(\sum_{i=1}^n \alpha_i y_i K(x_i, x) + b \right) \quad (2.9)$$

donde los coeficientes α_i se obtienen de la formulación dual. En la biblioteca *scikit-learn*, la clase SVC implementa esta optimización con parámetros clave:

- **C:** Controla el balance entre la amplitud del margen y los errores permitidos.
- **Kernel:** Define el tipo de transformación (lineal, polinomial o RBF).
- **Gamma:** Para *kernels* como RBF, afecta la flexibilidad del modelo y el alcance de influencia de un solo ejemplo de entrenamiento.

2.6.2 Flexibilidad mediante variables de holgura

Las variables de holgura juegan un papel importante en las máquinas de vectores de soporte de margen suave (*Soft Margin SVM*). Para comprender su importancia, es necesario considerar primero el concepto de margen suave.

En escenarios del mundo real, a menudo no es posible encontrar un hiperplano que separe perfectamente los datos. Aquí es donde entra en juego la SVM de margen suave, la cual permite cierta clasificación errónea de puntos de datos mediante la introducción de un término de penalización controlado por un hiperparámetro denominado parámetro de regularización (C). Un valor mayor de C indica una sanción mayor por clasificación errónea, lo que da como resultado un margen más estrecho; por el contrario, un valor más pequeño de C permite una mayor tolerancia al error, lo que conduce a un margen más amplio.

El papel de las variables de holgura (ξ_i) es manejar puntos de datos mal clasificados o que se encuentran dentro del margen. Estas variables representan la distancia de un punto que viola el margen desde su límite de clase correcto. En consecuencia, el problema de optimización se formula con restricciones de la siguiente forma:

$$\text{mín} \quad \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i \quad (2.10)$$

sujeto a las restricciones:

- $y_i(w^T x_i + b) \geq 1 - \xi_i$ para todo i .
- $\xi_i \geq 0$ para todo i .

En esta formulación, w representa el vector de peso, b es el término de sesgo, x_i es el vector de entrada, y_i es la etiqueta de clase correspondiente y ξ_i es la variable de holgura asociada con el i -ésimo ejemplo de entrenamiento.

El término $\frac{1}{2} \|w\|^2$ representa el margen y el término $C \sum \xi_i$ representa la penalización por clasificación incorrecta. Las restricciones aseguran que los puntos de datos se clasifiquen correctamente, con un margen de al menos $1 - \xi_i$. De esta manera, las variables de holgura permiten cierta flexibilidad, logrando un equilibrio entre maximizar el margen y minimizar el error.

Considerando un ejemplo donde las clases no son linealmente separables, una SVM de margen blando con la elección adecuada de estas variables puede encontrar un hiperplano funcional para la DDP con cierta tolerancia a la clasificación errónea, ayudando a cuantificar el alcance de las infracciones y ajustando el margen en consecuencia.

2.6.3 El truco kernel

El truco del kernel es un concepto fundamental en los algoritmos de la máquina de vectores de soporte (SVM) que permite el manejo de datos complejos transformándolos en un espacio de características de mayor dimensión. Esta técnica es particularmente útil cuando se trata de datos separables de forma no lineal, ya que permite que las SVM clasifiquen dichos datos de manera efectiva asignándolos implícitamente a un espacio de mayor dimensión. Para comprender el truco

del kernel, primero revisemos la idea básica detrás de las SVM. Las SVM son modelos de aprendizaje supervisado que tienen como objetivo encontrar un hiperplano óptimo que separe los puntos de datos que pertenecen a diferentes clases. En el caso de datos linealmente separables, un hiperplano lineal puede separar perfectamente las clases. Sin embargo, cuando los datos no son linealmente separables, las SVM emplean una función kernel para mapear los datos en un espacio de mayor dimensión donde la separación lineal es posible. Una función central es una función

matemática que toma dos vectores de entrada y calcula su similitud o producto interno en el espacio de características de dimensiones superiores. Permite que las SVM operen en el espacio de entrada original sin calcular explícitamente la transformación al espacio de dimensiones superiores. Esto es importante porque el cálculo explícito de la transformación podría resultar costoso o incluso imposible para ciertos tipos de transformaciones. Al usar el truco del kernel, las SVM pueden calcular de manera eficiente el límite de decisión en el espacio de características de mayor dimensión sin transformar explícitamente los datos. Esto se logra definiendo la función kernel para calcular implícitamente el producto interno entre los vectores de características transformados. El algoritmo SVM solo necesita calcular la función kernel para pares de vectores de entrada, en lugar de calcular la transformación para todos los puntos de datos individuales. Hay varios tipos de funciones de kernel que se usan comúnmente en las SVM, incluidas las funciones lineales, polinómicas, de base radial gaussiana (RBF) y kernels sigmoides. Cada función del núcleo tiene sus propias características y la elección del núcleo depende del problema específico y la naturaleza de los datos. Por ejemplo, el kernel lineal simplemente calcula el producto interno entre los vectores de entrada, realizando efectivamente una clasificación lineal en el espacio de entrada original. El núcleo polinomial eleva el producto interno a una cierta potencia, lo que permite límites de decisión no lineales. El kernel RBF mide la similitud entre dos vectores en función de su distancia euclidiana, lo que permite que las SVM capturen patrones complejos en los datos. El kernel sigmoide calcula la tangente hiperbólica del producto interno, lo que puede ser útil en ciertos escenarios.

2.6.4 Enfoque matemático

Para comenzar, el *kernel* es un modelo para abordar la no linealidad siendo este un algoritmo discriminativo. Pero, ¿cómo funciona? Partimos de una configuración de regresión simple:

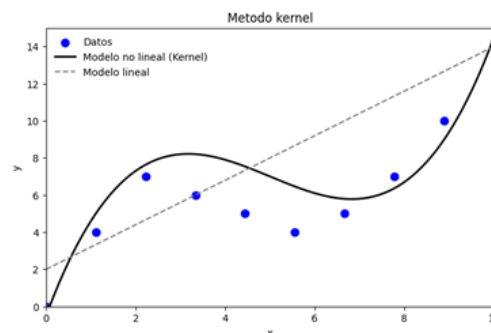


Figura 2.4: Gráfica que representa datos no lineales

Si observamos los datos como un modelo lineal, estaríamos cometiendo un error; podemos interpretar los datos mediante una función polinomial cúbica con el fin de ajustarlos. Sea $h(x)$ mi

modelo, siendo $x \in \mathbb{R}$ por simplicidad:

$$h(x) = \theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3$$

Teniendo así un modelo no lineal en x , pero sí lineal en θ . Esto es fundamental, ya que la linealidad en los parámetros es suficiente para aplicar técnicas conocidas. Podemos convertir esto a una Regresión Lineal estándar definiendo el vector de características:

$$\phi(x) = \begin{bmatrix} 1 \\ x \\ x^2 \\ x^3 \end{bmatrix}$$

Reescribiendo así el modelo no lineal a uno lineal:

$$h(x) = \theta^T \phi(x)$$

Con esto, el modelo es lineal en θ y lineal en las coordenadas $\phi(x)$. Antes la entrada era unidimensional, pero con $\phi(x)$ tenemos un vector de $p = 4$ dimensiones. Si nuestro dataset original era $\{x_1, x_2, \dots, x_n\}$, el nuevo dataset estará compuesto por $\{\phi(x_1), \phi(x_2), \dots, \phi(x_n)\}$.

Al aplicar el **descenso del gradiente** para minimizar la función de pérdida:

$$J(\theta) = \frac{1}{2} \sum_{i=1}^n (h(x_i) - y_i)^2$$

En el caso estándar, el algoritmo de actualización es:

$$\theta_{t+1} = \theta_t - \alpha \nabla J(\theta_t)$$

$$\theta_{t+1} = \theta_t - \alpha \sum_{i=1}^n (\theta_t^T \phi(x_i) - y_i) \phi(x_i)$$

¿Cuál es la dimensión de θ ? Es la misma que la de $\phi(x)$, es decir, p . Si p es muy alto (por ejemplo, con monomios de alto grado), el costo de operación $O(p)$ por cada actualización vuelve al algoritmo ineficiente.

Para acelerar esto, aplicamos el **truco del kernel**. Observamos que θ puede representarse como una combinación lineal de las características de entrenamiento:

$$\theta = \sum_{i=1}^n \beta_i \phi(x_i)$$

Donde β es un vector de n dimensiones (el número de datos). Si inicialmente $\theta_0 = 0$, en cada iteración t :

$$\theta_{t+1} = \sum_{i=1}^n \beta_i^{(t+1)} \phi(x_i)$$

Sustituyendo esto en la regla de actualización, podemos encontrar cómo evoluciona β sin necesidad de calcular θ explícitamente:

$$\beta_j^{(t+1)} = \beta_j^{(t)} - \alpha \left(\sum_{i=1}^n \beta_i^{(t)} \phi(x_i)^T \phi(x_j) - y_j \right)$$

Para que esto sea eficiente, introducimos la función Kernel $K(x, z) = \phi(x)^T \phi(z)$. Por ejemplo, si:

$$\phi(x) = \begin{bmatrix} x_1^3 \\ x_1^2 x_2 \\ \vdots \end{bmatrix}$$

El producto interno en un espacio de alta dimensión p es costoso, pero si existe una función $K(x, z)$

tal que:

$$K(x, z) = (x^T z + c)^d$$

Entonces calcular el producto interno toma tiempo $O(d)$ (dimensión original) en lugar de $O(p)$.

Método de kernel para regresión:

1. Precalcular la matriz de Kernel $K_{ij} = K(x_i, x_j)$ para todo $i, j = 1, \dots, n$. Esto toma $O(n^2 d)$.
2. Inicializar $\beta^{(0)} = 0$.
3. Bucle de actualización:

$$\beta_j^{(t+1)} = \beta_j^{(t)} - \alpha \left(\sum_{i=1}^n \beta_i^{(t)} K_{ij} - y_j \right)$$

Este algoritmo tiene una complejidad por iteración de $O(n^2)$, lo cual es mucho mejor que $O(np)$ siempre que $n < p$.

2.7 Estrategias de entrenamiento y validación

2.7.1 Manejo de clases desbalanceadas

El desbalance de clases ocurre cuando la distribución de las etiquetas de salida es significativamente asimétrica. Un modelo entrenado bajo estas condiciones tenderá a maximizar la exactitud global simplemente prediciendo la clase mayoritaria, ignorando los casos minoritarios (que suelen ser los de mayor interés, como el fraude o enfermedades raras).

- **Random Undersampling:** Consiste en eliminar aleatoriamente ejemplos de la clase mayoritaria para igualar la proporción de la minoritaria. Aunque reduce el tiempo de cómputo, existe el riesgo de descartar información valiosa que define los límites de decisión de la clase dominante.
- **SMOTE (Synthetic Minority Over-sampling Technique):** En lugar de duplicar registros existentes, SMOTE genera nuevas instancias sintéticas. El algoritmo selecciona un punto de la clase minoritaria, identifica sus k vecinos más cercanos y traza segmentos de línea entre ellos, creando nuevos puntos sobre dichos segmentos. Esto expande la región de decisión de la clase minoritaria de forma coherente.

2.7.2 Optimización de hiperparámetros

A diferencia de los parámetros (como los pesos θ que el modelo aprende por sí mismo), los hiperparámetros son configuraciones externas que definen el comportamiento del algoritmo de aprendizaje.

- **SVM y el parámetro C :** Regula el *trade-off* entre el margen de decisión y el error de clasificación. Un C pequeño permite un margen más amplio aceptando algunos errores (sesgo alto, varianza baja), mientras que un C grande intenta clasificar todos los puntos correctamente, arriesgando el sobreajuste (*overfitting*).
- **Naive Bayes y el suavizado de Laplace (α):** Evita el problema de la "probabilidad cero". Si una palabra no aparece en el entrenamiento para una clase específica, la probabilidad sería 0, anulando todo el producto. α añade un conteo pequeño a todas las frecuencias para garantizar estabilidad numérica.
- **Grid Search vs. Random Search:** Mientras que Grid Search evalúa todas las combinaciones posibles en un rango definido (exhaustivo pero lento), Random Search selecciona combinaciones al azar, lo que suele ser más eficiente para encontrar zonas de optimización en espacios de alta dimensión.

2.7.3 Validación cruzada (K-Fold Cross-Validation)

Para obtener una estimación robusta del rendimiento y evitar que la métrica dependa de una partición de datos "afortunada", se utiliza K-Fold. El dataset se divide en K subconjuntos iguales (folds). El proceso se repite K veces:

1. Se reserva el fold k para validación.
2. Se entrena el modelo con los $K - 1$ restantes.
3. Se calcula la métrica en el fold de validación.

Al final, el rendimiento reportado es el promedio de las K iteraciones:

$$E_{total} = \frac{1}{K} \sum_{i=1}^K E_i$$

2.8 Evaluación de la inteligencia del sistema

Para medir la eficacia real, especialmente en sistemas no balanceados, no basta con la exactitud. Debemos recurrir a la **Matriz de Confusión**, que clasifica los resultados en Verdaderos Positivos (VP), Falsos Positivos (FP), Verdaderos Negativos (VN) y Falsos Negativos (FN).

- **Accuracy (Exactitud):** Representa la fracción total de predicciones correctas. Es engañosa si las clases están desbalanceadas.

$$\text{Accuracy} = \frac{VP + VN}{VP + VN + FP + FN}$$

- **Precisión (Valor Predictivo Positivo):** Responde a: "¿De todas las veces que el sistema predijo 'Queja', cuántas realmente lo eran?". Es crítica cuando el costo de un Falso Positivo es alto.

$$\text{Precisión} = \frac{VP}{VP + FP}$$

- **Recall (Sensibilidad):** Responde a: "¿De todas las quejas reales que existían, cuántas logró detectar el sistema?". Es vital cuando no podemos permitirnos omitir casos (Falsos Negativos).

$$\text{Recall} = \frac{VP}{VP + FN}$$

- **F1-Score:** Es la media armónica de la precisión y el recall. Penaliza valores extremos, siendo una métrica excelente para encontrar un equilibrio entre ambas.

$$F1 = 2 \cdot \frac{\text{Precisión} \cdot \text{Recall}}{\text{Precisión} + \text{Recall}}$$

- **Curvas ROC y AUC:** La curva ROC (*Receiver Operating Characteristic*) grafica la Tasa de Verdaderos Positivos frente a la Tasa de Falsos Positivos al variar el umbral de clasificación. El área bajo la curva (AUC) mide la capacidad del modelo para distinguir entre clases; un AUC de 1.0 es perfecto, mientras que 0.5 equivale a una clasificación aleatoria.

Capítulo 3

Infraestructura y Tecnologías

3.1 Estudio de Arquitecturas de Software

3.1.1 Introducción a las Arquitecturas de Software

- Definición de arquitectura de software
- Importancia en el desarrollo de sistemas
- Factores de calidad:
 - Rendimiento
 - Escalabilidad
 - Mantenibilidad
- Clasificación general de arquitecturas:
 - Monolíticas
 - Distribuidas
 - Orientadas a servicios
 - Basadas en eventos

3.1.2 Arquitectura Monolítica

- Definición y estructura
- Funcionamiento interno
- Ventajas
- Desventajas
- Casos de uso

3.1.3 Arquitectura Orientada a Servicios (SOA)

- Definición y principios
- Enterprise Service Bus (ESB)
- Protocolos de comunicación:
 - SOAP
 - XML
- Ventajas
- Desventajas
- Diferencias con microservicios

3.1.4 Arquitectura de Microservicios

- Definición y características
- Principios fundamentales:
 - Independencia de despliegue
 - Alta cohesión
 - Bajo acoplamiento
- Descomposición funcional

- Comunicación entre servicios:
 - REST
 - JSON
 - API Gateway
- Patrones clave:
 - API Gateway
 - Service Discovery
 - Circuit Breaker
- Gestión de datos:
 - Base de datos por servicio
 - Base de datos compartida
- Ventajas
- Desventajas

3.1.5 Arquitectura de Microservicios en 3 Capas

- Capa de presentación
- Capa de lógica
- Capa de datos
- Flujo de petición
- Justificación del diseño

3.1.6 Arquitectura Dirigida por Eventos (Event-Driven)

- Concepto de eventos
- Productores y consumidores

- Message brokers:
 - RabbitMQ
 - Kafka
- Comunicación asíncrona
- Consistencia eventual
- Ventajas
- Desventajas

3.1.7 Arquitectura Serverless (FaaS)

- Definición
- Ejecución basada en eventos
- Plataformas:
 - AWS Lambda
 - Google Cloud Functions
- Características:
 - Escalabilidad automática
 - Pago por uso
 - Stateless
- Ventajas
- Desventajas

3.1.8 Arquitectura Hexagonal (Ports and Adapters)

- Definición
- Separación del dominio

- Puertos y adaptadores
- Independencia tecnológica
- Beneficios en pruebas

3.1.9 Comparación de Arquitecturas

- Escalabilidad
- Complejidad
- Acoplamiento
- Mantenibilidad
- Casos de uso

3.1.10 Selección de Arquitectura

- Criterios de selección
- Evaluación según contexto
- Justificación de la arquitectura elegida

Capítulo 4

Requerimientos del Sistema

4.1 Requerimientos Funcionales (RF)

ID	Nombre	Descripción
RF-01	Registro de Queja	El sistema debe permitir al quejoso llenar el formulario...
RF-02	Clasificación con PLN	El sistema debe sugerir la competencia usando IA...

4.2 Reglas de Negocio (RN)

ID	Descripción
RN-01	Solo miembros de la comunidad IPN con correo institucional pueden registrar quejas.

Pantallas y Mockups del Sistema

En este apartado se presentan los diseños de interfaz aprobados para la plataforma de la Defensoría de los Derechos Politécnicos. La estructura se divide por perfiles de usuario para reflejar el flujo operativo del sistema.

.1 Perfil: Quejoso

Este perfil corresponde a la comunidad politécnica (alumnos, docentes, administrativos). Las interfaces se centran en la accesibilidad y el seguimiento simple.

ID	Nombre de la Pantalla	Descripción
MQ-01	Landing Page / Inicio	Acceso público e información general de la DDP.
MQ-02	Formulario de Registro	Captura de datos de la queja con asistencia de PLN.
MQ-03	Consulta de Estatus	Seguimiento mediante folio digital y token.

Tabla 1: Inventario de Mockups - Perfil Quejoso.

Figura 1: MQ-01: Pantalla de inicio y selección de trámite.

.2 Perfil: Recepcionista

Interfaces diseñadas para la gestión administrativa inicial y asignación de folios físicos a digitales.

ID	Nombre de la Pantalla	Descripción
MR-01	Dashboard de Recepción	Vista general de solicitudes pendientes de radicación.
MR-02	Registro Manual	Interfaz para digitalizar quejas recibidas de forma presencial.

Tabla 2: Inventario de Mockups - Perfil Recepcionista.

.3 Perfil: Oficina de Primer Contacto

Interfaces especializadas en la clasificación técnica y jurídica preliminar mediante el asistente de IA.

ID	Nombre de la Pantalla	Descripción
MPC-01	Módulo de Clasificación	Interfaz con sugerencias del motor de PLN para competencia.
MPC-02	Bandeja de Turno	Gestión de expedientes para su envío a abogados asesores.

Tabla 3: Inventario de Mockups - Perfil Primer Contacto.

.4 Perfil: Subdefensores

Interfaces de alta prioridad para el análisis sustantivo, emisión de acuerdos y resoluciones finales.

ID	Nombre de la Pantalla	Descripción
MSD-01	Gestión de Expediente Único	Vista integral de documentos, evidencias y cronología.
MSD-02	Editor de Resoluciones	Herramienta para la carga y firma digital de documentos oficiales.

Tabla 4: Inventario de Mockups - Perfil Subdefensores.

Casos de Uso

A continuación se detallan los diagramas de casos de uso que describen las interacciones entre los diferentes actores (Abogados, Quejosos, Administradores) y el sistema.

Figura 2: Diagrama general de casos de uso del sistema DDP.

Bibliografía

- [1] Instituto Politécnico Nacional, “*Ley Orgánica del Instituto Politécnico Nacional*”, Diario Oficial de la Federación, Ciudad de México, México. [En línea]. Disponible: <https://www.ipn.mx/assets/files/defensoria/docs/manuales/ley-org%C3%A1nica-ipn.pdf>
- [2] Instituto Politécnico Nacional, “*Reglamento Interno del Instituto Politécnico Nacional*”, Gaceta Politécnica, Ciudad de México, México. [En línea]. Disponible: <https://www.ipn.mx/assets/files/defensoria/docs/manuales/reglamento-interno.pdf>
- [3] Instituto Politécnico Nacional, “*Acuerdo de creación de la Defensoría de los Derechos Politécnicos*”, Gaceta Politécnica, no. 622 (Extraordinaria), pp. 3-7, ene. 31, 2006. [En línea]. Disponible: <https://www.ipn.mx/assets/files/imageninstitucional/docs/gaceta-extraordinaria/2006/01/622-31-enero-gaceta-extra-.pdf>
- [4] Instituto Politécnico Nacional, “*Manual de Organización de la Defensoría de los Derechos Politécnicos*”, Dirección de Planeación y Organización, Ciudad de México, México, oct. 2022. [En línea]. Disponible: <https://www.ipn.mx/assets/files/defensoria/docs/manuales/moddp2022.pdf>
- [5] Instituto Politécnico Nacional, “*Manual de Procedimientos de la Defensoría de los Derechos Politécnicos*”, Dirección de Planeación y Organización, Ciudad de México, México, oct. 2024. [En línea]. Disponible: <https://www.ipn.mx/assets/files/defensoria/docs/manuales/mp-ddp.pdf>
- [6] T. Mikolov, K. Chen, G. Corrado, y J. Dean, “*Efficient Estimation of Word Representations in Vector Space*”, en Proceedings of Workshop at ICLR, 2013. [En línea]. Disponible: <https://arxiv.org/abs/1301.3781>
- [7] Fortres Grand Corporation, “*Bully Incident Reporting & Tracking Software | BullyButton.com*”, 2018. [En línea]. Disponible en: <http://bullybutton.fortresgrand.com/>. [Accedido: 16-sep-2025].

- [8] Mejora tu Escuela, “*Ventanilla Escolar*”, 2018. [En línea]. Disponible en: <http://www.mejoratuescuela.com/ventanilla-escolar/#/>. [Accedido: 16-sep-2025].
- [9] Instituto Politécnico Nacional, “*Alerta*”, 2018. [En línea]. Disponible en: <https://www.alerta.ipn.mx/>. [Accedido: 16-sep-2025].
- [10] Universidad Nacional Autónoma de México, “*Denuncia en Línea - Defensoría de los Derechos Universitarios*”, 2024. [En línea]. Disponible en: <https://www.defensoria.unam.mx/>. [Accedido: 17-feb-2026].
- [11] Secretaría del Trabajo y Previsión Social, “*Procuraduría Federal de la Defensa del Trabajo (PROFEDET)*”, 2024. [En línea]. Disponible en: <https://www.gob.mx/profedet>. [Accedido: 17-feb-2026].